FINANCIAL RAG CHATBOT

Ingest

Q&A

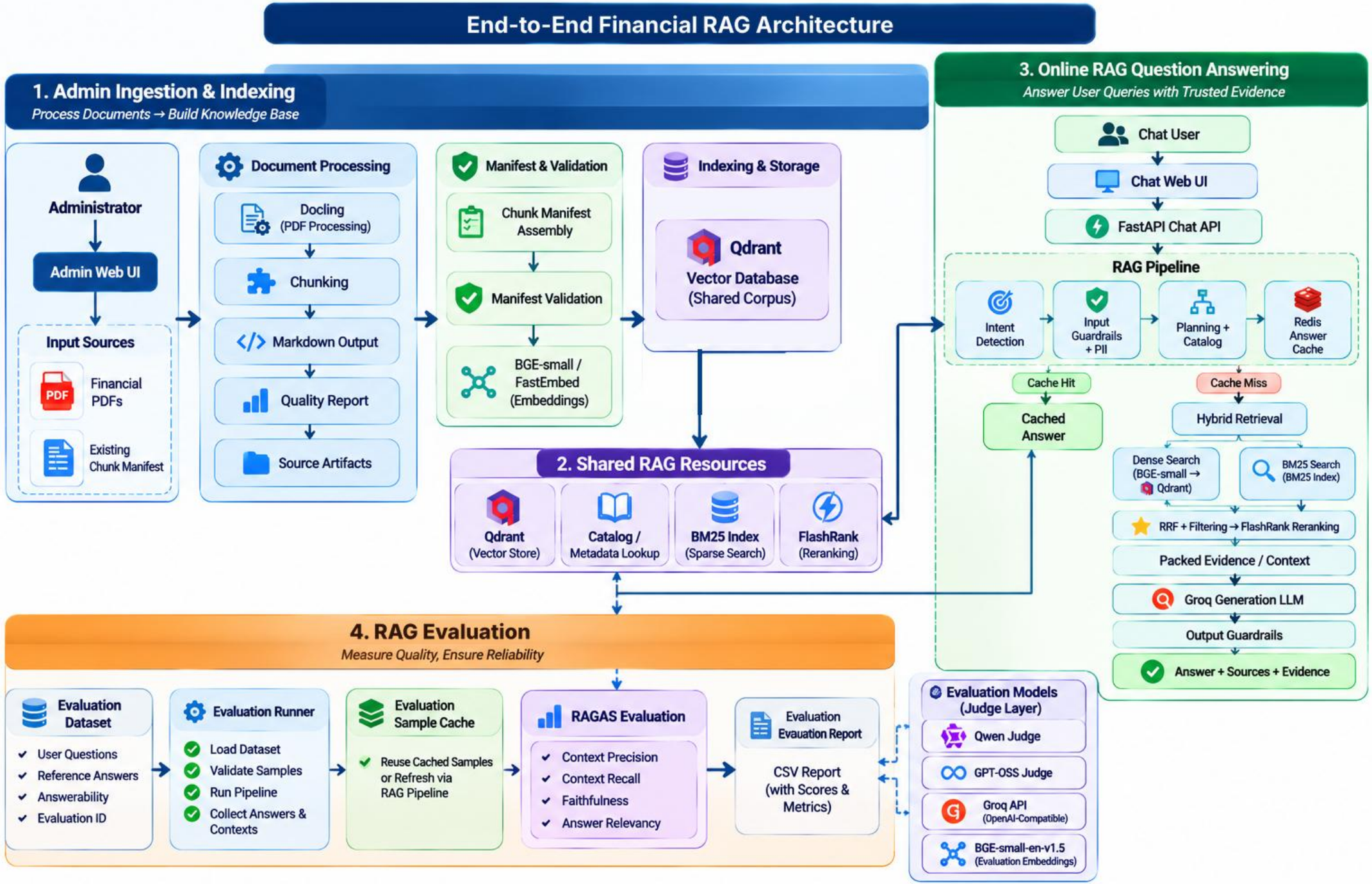
Evaluate

Presented by,
Nithin Shetty M

Index

What this deck covers, in the order of a document's life

1	System architecture	Slide 3
3	Online Q&A architecture	Slide 8
4	Routing, input guardrails, planning and cache	Slide 9 – 10
5	Retrieval, reranking and generation	Slide 11 – 12
6	Output guardrails and grounding	Slide 13
7	Evaluation: architecture and RAGAS metrics	Slide 14 – 15
8	Serving, observability and tech stack	Slide 16
9	References and further reading	Slide 17


End-to-End Financial RAG Architecture
1. Admin Ingestion & Indexing
Process Documents → Build Knowledge Base
Administrator
Admin Web UI
Input Sources
Financial PDFs
Existing Chunk Manifest
Document Processing
Docling (PDF Processing)
Chunking
Markdown Output
Quality Report
Source Artifacts
Manifest & Validation
Chunk Manifest Assembly
Manifest Validation
BGE-small / FastEmbed (Embeddings)
Indexing & Storage
Qdrant
Vector Database (Shared Corpus)
2. Shared RAG Resources
Qdrant (Vector Store)
Catalog / Metadata Lookup
BM25 Index (Sparse Search)
FlashRank (Reranking)
3. Online RAG Question Answering
Answer User Queries with Trusted Evidence
Chat User
Chat Web UI
FastAPI Chat API
RAG Pipeline
Intent Detection
Input Guardrails + PII
Planning + Catalog
Redis Answer Cache
Cache Hit
Cached Answer
Cache Miss
Hybrid Retrieval
Dense Search (BGE-small → Qdrant)
BM25 Search (BM25 Index)
RRF + Filtering → FlashRank Reranking
Packed Evidence / Context
Groq Generation LLM
Output Guardrails
Answer + Sources + Evidence
4. RAG Evaluation
Measure Quality, Ensure Reliability
Evaluation Dataset
User Questions
Reference Answers
Answerability
Evaluation ID
Evaluation Runner
Load Dataset
Validate Samples
Run Pipeline
Collect Answers & Contexts
Evaluation Sample Cache
Reuse Cached Samples or Refresh via RAG Pipeline
RAGAS Evaluation
Context Precision
Context Recall
Faithfulness
Answer Relevancy
Evaluation Evaluation Report
CSV Report (with Scores & Metrics)
Evaluation Models (Judge Layer)
Qwen Judge
GPT-OSS Judge
Groq API (OpenAI-Compatible)
BGE-small-en-v1.5 (Evaluation Embeddings)

Parsing with Docling: structure first

parse_pdf.py: FastDoclingChunker turns a PDF into a structured DoclingDocument

ACCURATE
TableFormer mode + PDF cell matching

> 20%
pages without text triggers OCR fallback

4
artifacts saved per document

0
silent truncation of text

Layout and tables

- Tables read from structured TableItem: cells, row/col spans, header flags
- Cells expanded to a dense grid
- Reading order from bounding boxes
- No regex on flattened text

Scanned PDFs

- ocr_mode: auto, on or off
- Auto: re-convert with OCR when over 20% of pages lack text
- Falls back to the non-OCR result if OCR fails
- Born-digital PDFs stay fast

Cleanup and QA

- Boilerplate and glyph-only noise removed
- Fragments (under 35 chars or 6 words) are merged, not dropped
- Quality report JSON per document

Chunking: three chunk types, loss-free

Docling HybridChunker, tokenized with the same bge-small tokenizer used for embedding

text

- HybridChunker, heading-contextualized
- Budget: 500 tokens minus the Document / Page / Section prefix
- merge_peers disabled

table

- Whole table as a Markdown block
- Caption, columns, row labels kept in metadata
- Good for context and citation

table_fact

- Atomic multi-year row facts
- Metadata: fiscal_years, row_group, financial_metric
- Primary retrieval target for numeric questions

Loss-free splitting

- Oversized text is split by lines, then sentences, then words, then raw token slices
- Nothing is cut to fit

Artifacts per document

- _manifest.json (chunks), _document.md
- _quality.json, _docling.json (re-chunk without re-parsing)

Indexing into Qdrant

seed_qdrant.py + Streamlit admin UI (parse tab, upload-manifest tab)

Validate manifest › **Embed (bge-small, batch 32)** › **Upsert points** › **Create payload indexes** › **Delete orphan points**

Vector store

- Collection financial_rag_analytics
- Named vector fastembed-BAAI/bge-small-en-v1.5
- 384 dimensions, cosine distance
- Local path or remote QDRANT_URL

Idempotent writes

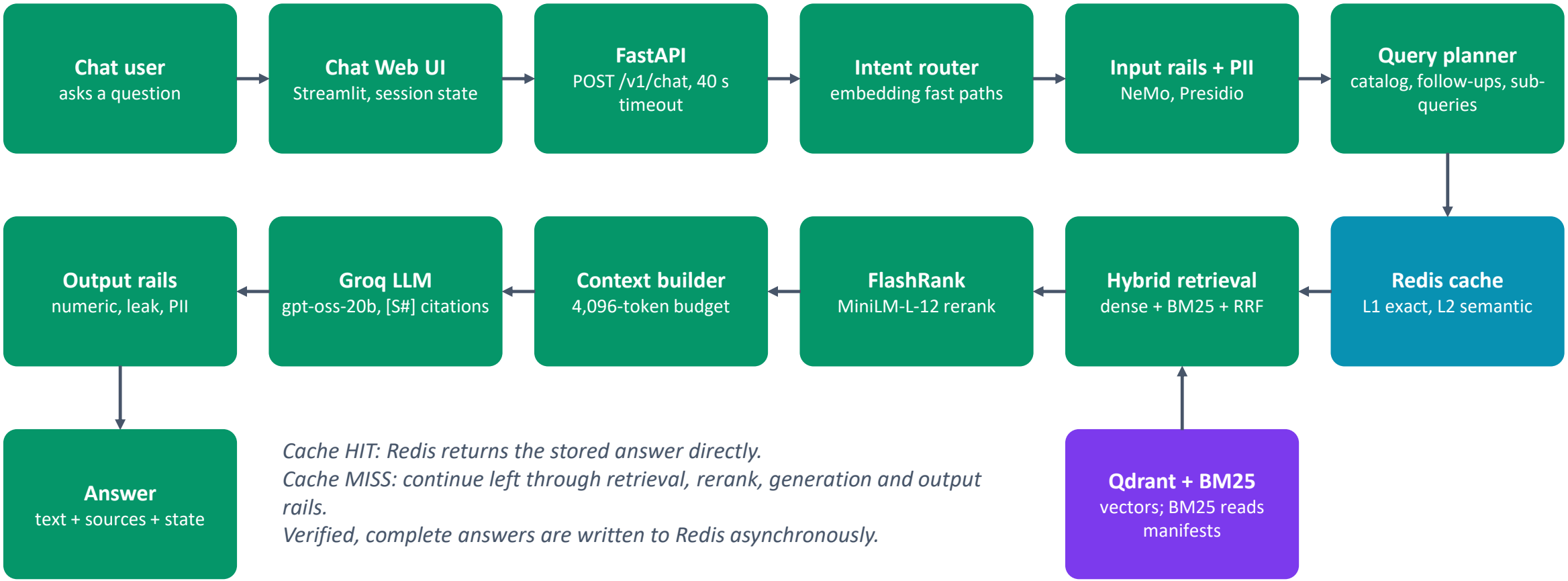
- Point ID = uuid5(doc_id : chunk_id)
- Re-indexing overwrites, never duplicates
- Points of that document absent from the new manifest are deleted
- Company and year inferred if missing

Filters and safety

- Payload indexes: document_id, chunk_type, company, fiscal_year, fiscal_years
- Manifest file is kept on disk for the runtime BM25 search
- Recreate-collection needs explicit confirmation in the UI

Architecture: online question answering

FastAPI request path; cache hit exits early, miss continues to retrieval



Intent routing and input guardrails

Fast, data-driven filters before any expensive call

Embedding intent router

- Labelled examples (intent_examples.yml) embedded once at startup with bge-small
- Cosine similarity to nearest example per class
- Fast labels: smalltalk, elaboration, reference, summary, out_of_scope
- Fires only if score 0.85 or more, margin 0.03 or more, and short (12 words; 40 for out-of-scope)
- Otherwise falls through to the LLM router: a miss costs latency, not correctness
- Text containing PII never takes a fast path

NeMo Guardrails as a policy layer

- check_async() on input rails only; the RAG pipeline still owns generation
- Injection heuristics + self_check_input LLM check (Groq)
- PII masking via Presidio, regex fallback
- 8 s timeout; on NeMo failure, fail-open to deterministic rules (configurable)
- Blocked requests get the Colang message or a scope refusal; raw PII is never logged

Query planning, catalog and two-tier cache

Cached answers are a pure function of (standalone question, scope, corpus)

Planner and catalog

- Catalog = documents in Qdrant (TTL 300 s, manifest fallback)
- LLM router sees known companies; outputs intent, entities, years, metrics, sub-queries, search_query
- Resolves follow-ups: none, reference (rewrite to standalone), elaboration (needs previous answer)
- SessionState (company, year, metric, document) carried across turns
- Unknown company returns a clear not-found message

Redis cache

- L1: exact match. L2: semantic, cosine distance 0.04 or less, then Jaccard word overlap 0.7 or more
- Scope key = intent + entities + years + corpus_version (+ hash of previous answer for elaboration)
- Elaboration uses L1 only
- TTL: 24 h answers, 7 d greetings
- Never cached: chat summary, general chat, incomplete answers; writes are fire-and-forget

Hybrid retrieval: dense + BM25 with RRF

hybrid_search_balanced(): per-entity quotas keep multi-company questions fair

Sub-queries per entity › **Dense bge-small + BM25 (parallel)** › **Relax filters if under 5 hits** › **RRF fusion** › **Entity quota to 30 candidates**

k = 60	30	8	5
RRF constant	rerank candidates	max concurrent searches	min strict hits before relaxing

Filtering

- Metadata filters: document_ids, company, fiscal years
- If strict results are thin, retry with document filter only

Fusion

- Score = sum of 1 / (60 + rank) across rankings
- Rank-based, so no score normalization between cosine and BM25

Balance

- Quota = total / entities, spare slots filled by best leftovers
- Failures in one retriever return empty, not an error

Reranking, context packing and generation

From 30 candidates to a small, cited, token-budgeted context

FlashRank rerank

- ms-marco-MiniLM-L-12-v2, CPU only
- rerank_balanced with entity-aware queries (company + metrics + years)

Final-k policy

- Default 5, cap 10
- At least 3 chunks per company
- +3 for elaboration follow-ups

Context builder

- Blocks tagged [S1], [S2]... with file, page, section
- Stops at 4,096 tokens (token-counted)
- source_map powers citations

System prompt + context › **Groq gpt-oss-20b** › **Output guardrails** › **Strict re-prompt if figures ungrounded**

Key rotation: a fallback Groq API key and chat_with_retry handle rate limits. Conversation history is sanitized and capped at 6 turns.

Output guardrails and grounding

Financial answers must be traceable to the retrieved evidence

Numeric grounding

- Every figure in the answer must appear in the context
- Ungrounded ratio over 0.5 blocks the answer in fallback mode
- Numeric block triggers one strict retry

Leak and PII checks

- Prompt-leak detection
- Masks email, card, SSN, IBAN, bank, Aadhaar, PAN
- Phone numbers allowed in output: filings contain IR phones

Completeness

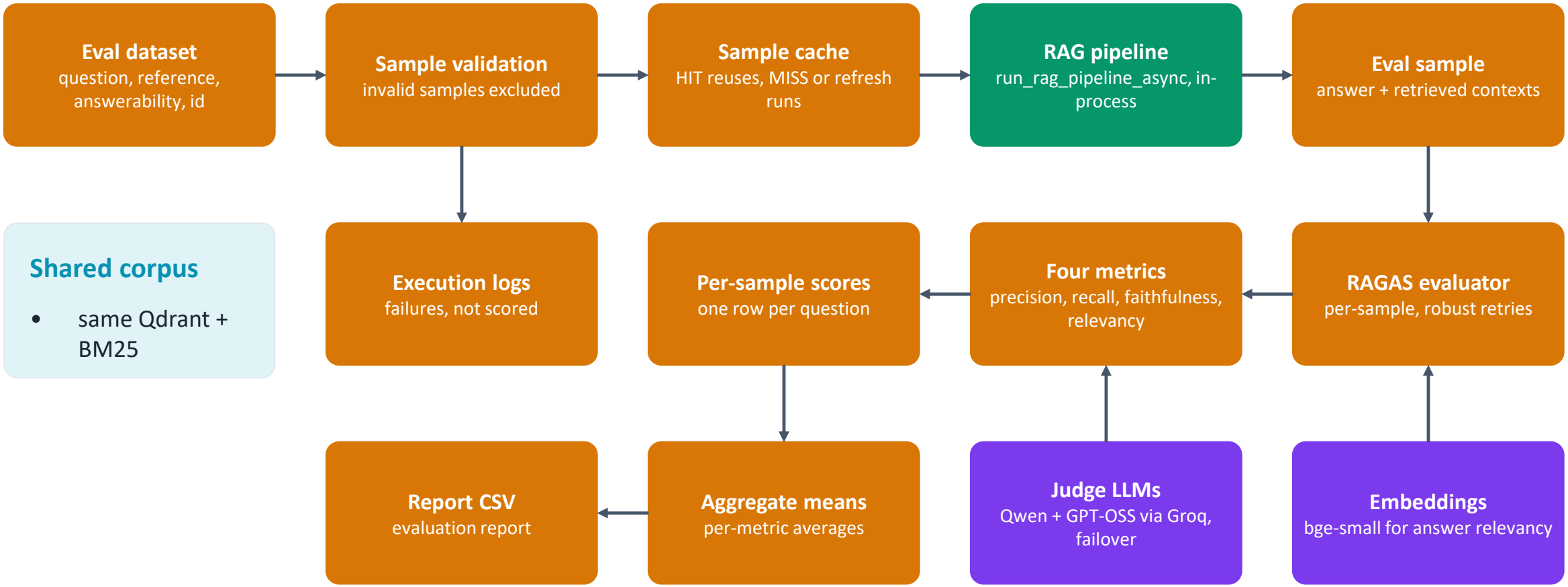
- Every requested company must be in the evidence
- Missing company: prefix note, answer not cached
- Blocked answers are never cached

Response contract (ChatResponse)

- answer, retrieved_chunks (page, type, rrf_score, final_score), source_map, updated SessionState, intent, search_query

Architecture: offline RAGAS evaluation

Reuses the production pipeline in-process, so scores reflect the real system



Offline evaluation with RAGAS

ragas4.py runs the real pipeline in-process and scores each sample

Dataset (question, reference, id) › Validate samples › Cache hit or fresh pipeline run › RAGAS scoring › Per-sample CSV + means

Metric	Question it answers
Context precision	Are the relevant chunks ranked high?
Context recall	Does the context cover the reference answer?
Faithfulness	Is every claim supported by the context?
Answer relevancy	Does the answer address the question?

Judge infrastructure

- Two judge LLMs (Qwen, GPT-OSS) through Groq's OpenAI-compatible API
- Custom httpx transport: dual keys, model failover, retry-after backoff
- Up to 5 contexts of 1,500 chars per sample; pipeline concurrency 2
- bge-small embeddings for answer relevancy
- Sample cache avoids re-running generation; refresh flag forces it

Serving, observability and tech stack

Designed to fail soft and be debuggable in production

Serving (FastAPI)

- /health, /health/ready (503 while models warm up), /v1/chat
- Warmup preloads embeddings, BM25, reranker, router, guardrails, cache
- 40 s request timeout returns 504
- Streamlit chat UI checks readiness, shows source chunks

Observability (Logfire)

- FastAPI, httpx and pydantic instrumented
- Spans per stage with intent, cache tier, tokens, guardrail events
- Query PII-masked before logging

Docling | **bge-small / FastEmbed** | **Qdrant** | **BM25** | **FlashRank** | **Groq gpt-oss-20b** | **NeMo Guardrails**

Presidio | **Redis** | **FastAPI** | **Streamlit** | **RAGAS** | **Logfire**

Next: tune intent and cache thresholds from logs, grow the eval set, add more filings.

References and further reading

Docling Technical Report arXiv:2408.09869; github.com/docling-project/docling
TableFormer: table structure with transformers arXiv:2203.01017
RAG for knowledge-intensive NLP (Lewis et al.) arXiv:2005.11401
C-Pack / BGE embeddings arXiv:2309.07597; huggingface.co/BAAI/bge-small-en-v1.5
Reciprocal Rank Fusion (Cormack, Clarke, Buettcher) SIGIR 2009
BM25: The Probabilistic Relevance Framework Robertson and Zaragoza, 2009
RAGAS: automated evaluation of RAG arXiv:2309.15217; docs.ragas.io
NeMo Guardrails arXiv:2310.10501; github.com/NVIDIA/NeMo-Guardrails
Docs: Qdrant, FlashRank, Presidio, Logfire qdrant.tech/documentation; github.com/PrithivirajDamodaran/FlashRank; microsoft.github.io/presidio; logfire.pydantic.dev/docs
Video talks YouTube: search "Docling IBM Research", "hybrid search RRF Qdrant", "RAGAS evaluation tutorial"